UNIVERSIDAD NACIONAL DE COLOMBIA (SEDE MANIZALES)

Fundamentación, Arquitectura y Selección Tecnológica: Pipeline de Visión Artificial en ROS 2 Jazzy y Gazebo Harmonic

Proyecto Propuesta 2: Robot autónomo de inspección con visión artificial (100% Simulación con TurtleBot3, OpenCV y ArUco/HSV)

Avance 1 (Definición y Diseño): Problemática • Propuesta • Selección de Cámaras • Entorno y Librerías • Evidencia en Proyectos Similares

1. Planteamiento del Problema, Propuesta y Justificación para la Sustentación

En el desarrollo de un proyecto de robótica móvil, el éxito de la sustentación final depende de saber justificar **por qué se eligió cada herramienta, cómo se estructuró la arquitectura antes de programar y cómo se anticiparon las restricciones de la plataforma**. Este documento establece la línea base conceptual y técnica del proyecto.

1.1 Problemática a Resolver

En entornos industriales, energéticos y de infraestructura crítica, la supervisión rutinaria de tableros de control, indicadores luminosos o estaciones de trabajo suele realizarse mediante rondas manuales. Esto expone al personal a factores de riesgo, introduce subjetividad en la lectura y carece de un respaldo digital auditable. Sin embargo, al automatizar esta tarea mediante robótica móvil en el ámbito académico o industrial, se presenta una limitación recurrente: **los robots se limitan a ejecutar un "patrullaje ciego" (llegar a coordenadas X, Y), sin garantizar que el panel objetivo quede dentro del campo visual de la cámara, sufriendo falsos positivos por analizar un único fotograma en movimiento y sin distinguir entre un estado operativo normal y un fallo de captura**.

1.2 Nuestra Propuesta de Solución

Diseñar e implementar un sistema de inspección autónoma 100% simulado sobre **Ubuntu 24.04 LTS**, **ROS 2 Jazzy Jalisco** y **Gazebo Harmonic**, utilizando como base el robot diferencial **TurtleBot3** equipado con una cámara RGB frontal. El robot recorrerá tres estaciones de inspección (**P1, P2 y P3**), ajustará su orientación final (ángulo *yaw*) para enfrentar cada panel, detendrá su marcha para estabilizar la captura y ejecutará algoritmos de **Visión Artificial Clásica con OpenCV**: lectura de marcadores fiduciales **ArUco** para identificar inequívocamente la estación y segmentación en el espacio de color **HSV** para diagnosticar estados operativos (Verde = Normal, Rojo = Alerta). Finalmente, decidirá por votación temporal sobre múltiples capturas y registrará un informe trazable (CSV/JSON) junto con las imágenes anotadas como evidencia.

1.3 Justificación Metodológica: ¿Por qué iniciar por el Pipeline de Visión Artificial?

Para sustentar nuestro proyecto, nuestro equipo prioriza la arquitectura del sistema de visión artificial antes de abordar la navegación autónoma (Nav2) por tres razones de ingeniería:

- **Superación de la ruptura tecnológica entre Gazebo Classic y Gazebo Harmonic:** Gran parte de la literatura previa en ROS utiliza el plugin obsoleto `libgazebo_ros_camera.so` de Gazebo Classic 11. En Ubuntu 24.04 con ROS 2 Jazzy, el simulador oficial es **Gazebo Harmonic** [4], donde los sensores operan bajo una arquitectura desacoplada gobernada por el sistema `gz::sim::systems::Sensors` y requieren puentes dedicados (`ros_gz_image` / `ros_gz_bridge`) [2], [3]. Comprender y diseñar primero esta interfaz asegura que el robot cuente con percepción visual confiable antes de moverse.
- **Principio de diseño modular y aislamiento de fallos:** Separar el problema de percepción (cámara + OpenCV) del problema de control y localización (Nav2 + AMCL) permite validar las condiciones de iluminación, distancia focal y diccionarios ArUco sin que errores de odometría contaminen las pruebas iniciales [1].
- **Presupuesto computacional en Máquina Virtual (VirtualBox):** El renderizado 3D de cámaras virtuales demanda recursos significativos de CPU/GPU. Dimensionar primero la cantidad de cámaras, su resolución y su frecuencia de muestreo evita saturar la máquina virtual cuando posteriormente se integre la planificación de rutas [1].

2. Entorno de Trabajo y Ecosistema de Librerías Requeridas

A partir de las especificaciones del curso y la revisión de proyectos equivalentes en ROS 2 Jazzy, se definió la pila tecnológica (*software stack*) estrictamente necesaria para construir el proyecto sin conflictos de versiones ni dependencias obsoletas.

2.1 Entorno Base de Ejecución

- **Sistema Operativo Anfitrión / Invitado:** Ubuntu 24.04 LTS (Noble Numbat) ejecutado sobre máquina virtual VirtualBox (64 bits).
- **Middleware Robótico:** ROS 2 Jazzy Jalisco (distribución LTS nativa para Ubuntu 24.04).
- **Simulador Físico y Sensorial:** Gazebo Harmonic (`gz-sim8`), utilizando el motor de renderizado gráfico `ogre2` para simulación lumínica y de cámaras [3].
- **Lenguaje de Programación:** Python 3.12 (soportado nativamente por `rclpy` en ROS 2 Jazzy).

2.2 Librerías y Paquetes Necesarios para la Implementación

La siguiente tabla detalla cada librería y paquete requerido, la capa arquitectónica a la que pertenece y la justificación técnica de su uso dentro de nuestra solución:

Librería / Paquete	Capa del Sistema	Rol y Justificación en el Proyecto
`gz-sim8` y `gz-sensors8`	Simulador Gazebo Harmonic	Proporciona el plugin de sistema `gz::sim::systems::Sensors` encargado de procesar las etiquetas `<sensor type="camera">` del archivo SDFormat y publicar el flujo visual en Gazebo Transport [3].
`ros-jazzy-ros-gz-bridge` y `ros-jazzy-ros-gz-image`	Puente de Comunicaciones (Middleware)	Reemplaza a los antiguos plugins de Gazebo Classic. Conecta los tópicos internos de Gazebo (`gz.msgs.Image` y `gz.msgs.CameraInfo`) con los tópicos de ROS 2 (`sensor_msgs/msg/Image` y `CameraInfo`) [2].
`ros-jazzy-cv-bridge` (`cv_bridge`)	Adaptador ROS 2 ↔ OpenCV	Librería puente esencial que deserializa los mensajes binarios de imagen de ROS 2 y los convierte en matrices numéricas `numpy.ndarray` en formato de canales `BGR8` listas para ser procesadas por OpenCV.
`python3-opencv` (OpenCV 4.x + `cv2.aruco`)	Procesamiento de Visión Clásica	Provee los algoritmos de visión artificial clásica: conversión de espacios de color (BGR a Escala de Grises y HSV), filtrado morfológico, contornos y el submódulo `cv2.aruco` para lectura de IDs y esquinas [7].
`python3-numpy`	Cálculo Matricial	Manipulación rápida de matrices de píxeles, definición de rangos de umbralización HSV (límites inferiores y superiores) y cálculo de proporciones de área activa.
`ros-jazzy-rqt-image-view` y `ros-jazzy-rviz2`	Inspección y Diagnóstico Visual	Herramientas gráficas de ROS 2 para auditar en vivo el flujo crudo (`/camera/image_raw`) y las imágenes de evidencia anotadas por nuestro nodo.

3. Estudio de las Cámaras en Gazebo Harmonic: Selección, Ubicación y Herramientas

Uno de los aspectos más importantes al diseñar el sistema de percepción es comprender qué tipos de cámaras ofrece Gazebo Harmonic, cuántas requiere realmente nuestro robot, dónde debe instalarse y qué herramientas existen dentro del simulador para localizarla y verificar su funcionamiento.

3.1 ¿Qué tipos de cámaras existen en Gazebo Harmonic y cuál elegimos?

La biblioteca `gz-sensors` de Gazebo Harmonic ofrece tres tipos principales de sensores visuales en SDFormat [3]. Evaluar sus diferencias nos permite justificar nuestra elección:

Tipo de Sensor en SDF	Datos que Genera	Costo en VirtualBox	Decisión para el Proyecto
Cámara RGB Monocular (`type="camera"`)	Imagen 2D a color (`R8G8B8`) + matriz de calibración intrínseca (`CameraInfo`).	**Bajo / Moderado** (Ideal para VM).	**SELECCIONADA:** Cumple el 100% de los requisitos para ArUco y paneles HSV.
Cámara de Profundidad (`type="depth_camera"`)	Mapa de distancias por píxel (`R_FLOAT32`) sin información de color RGB.	Alto.	**Descartada:** No permite segmentar colores HSV (rojo/verde) ni leer texturas ArUco.
Cámara RGB-D (`type="rgbd_camera"`)	Imagen a color + mapa de profundidad + nube de puntos 3D (`PointCloud2`).	**Muy Alto** (Riesgo de caída de FPS en VM).	**Descartada:** Sobredimensionada; la distancia ArUco puede estimarse con `CameraInfo` [1].

3.2 ¿Cuántas cámaras necesitamos y qué modelo de TurtleBot3 usamos como referencia?

- **Cantidad de cámaras:** Requerimos **únicamente una (1) cámara RGB frontal monocular**. Agregar cámaras laterales o traseras multiplicaría el ancho de banda del puente `ros_gz_bridge` y el consumo de CPU en VirtualBox sin aportar valor, dado que el robot orientará su chasis (control de ángulo *yaw*) hacia cada panel antes de capturar [1].
- **Modelo de referencia en TurtleBot3:** Dentro de la familia oficial de ROBOTIS existen los modelos *Burger* (solo LiDAR e IMU) y *Waffle Pi* (que integra una cámara frontal Raspberry Pi además del LiDAR 360°) [4]. Nuestra configuración toma como referencia la geometría del **TurtleBot3 Waffle Pi**, configurando una resolución equilibrada de `640×480 píxeles` a una tasa de `15 Hz`.

3.3 ¿Dónde se ubica la cámara en el robot y cómo se orienta?

La ubicación espacial del sensor dentro del archivo de descripción del robot (SDF/URDF) responde a criterios geométricos y ópticos estrictos:

- **Posición física en el chasis (`camera_link`):** Se instala en el **frente del robot**, desplazada hacia adelante en el eje longitudinal (`x ≈ 0.069 m a 0.070 m` respecto al centro de la base) y a una altura intermedia (`z ≈ 0.107 m a 0.140 m` del suelo). Esta posición es estratégica porque ubica el lente justo por debajo del plano de barrido de la torreta del LiDAR (`base_scan`), evitando que el propio chasis del robot obstruya la imagen.
- **Diferencia entre el eslabón mecánico (`camera_link`) y el marco óptico (`camera_optical_frame`):**

- ◦ En el estándar de robótica móvil de ROS (REP-103), el soporte físico `camera_link` sigue los ejes **FLU**: **X hacia adelante**, **Y hacia la izquierda** y **Z hacia arriba**.
  - ◦ Sin embargo, OpenCV y el mensaje `CameraInfo` operan bajo la convención óptica de cámaras (**RDF**): el eje **Z apunta hacia adelante** (saliendo del lente hacia la escena), el eje **X apunta a la derecha** de la imagen (columnas de píxeles) y el eje **Y apunta hacia abajo** (filas de píxeles). Comprender esta transformación evita errores al calcular la orientación o distancia hacia un marcador ArUco [1], [7].
- **Alineación con las 3 estaciones de inspección (P1, P2 y P3):** Con un campo de visión horizontal (`horizontal_fov`) de `1.085 radianes` (~62.2°) y un rango de enfoque desde `0.05 m` hasta `8.0 m` , los paneles ArUco y de color en el mundo simulado deben colocarse a una altura baja-media (centro del panel entre `0.15 m` y `0.25 m` del piso), deteniendo el robot a una distancia de entre `0.50 m` y `1.00 m` frente a cada estación.

3.4 Herramientas dentro de Gazebo Harmonic para encontrar y auditar la cámara

En el entorno gráfico y de consola de Gazebo Harmonic disponemos de cuatro herramientas nativas para localizar la cámara y verificar qué está viendo antes de conectar cualquier script de Python:

1. **Entity Tree (Árbol de Entidades) y Component Inspector:** Ubicado en el panel lateral izquierdo de Gazebo Harmonic. Al desplegar el modelo del robot y hacer clic en el eslabón `camera_link` → sensor `camera` , muestra sus propiedades activas: resolución, tasa de refresco (`update_rate`), ángulo de visión y nombre del tópico publicado.
2. **Plugin gráfico *Image Display* (Visor de Cámara Interno):** Desde el menú superior derecho de plugins de Gazebo Harmonic, se puede agregar el panel **Image Display**. Permite seleccionar el tópico `/camera/image_raw` y observar en una subventana dentro del propio simulador exactamente lo que captura el lente del robot en tiempo real.
3. **Plugin *Topic Viewer* y herramienta de consola `gz topic` :** Permiten listar todos los canales internos activos en Gazebo Transport (por ejemplo, verificando que existan `/camera/image_raw` y `/camera/camera_info`).
4. **Control de Ejecución y Sistema Global `Sensors` :** En Gazebo Harmonic, las cámaras solo generan datos si se cumplen dos condiciones en el entorno: que el archivo del mundo cargue el plugin global `gz::sim::systems::Sensors` y que la simulación esté en reproducción activa (botón **Play** ▶ presionado en la barra inferior) [3].

> **Argumento**
>
> Demostrar que sabemos inspeccionar la cámara primero con *Image Display* dentro de Gazebo Harmonic y luego a través del puente `ros_gz_bridge` hacia ROS 2 demostrando un dominio real de la arquitectura por capas: **Capa 1 (Física/Renderizado en Gazebo) → Capa 2 (Transporte/Bridge) → Capa 3 (Percepción/OpenCV en ROS 2)**.

4. Evidencias Basadas en Proyectos Similares y Estado del Arte Práctico

Nuestras decisiones sobre el tipo de cámara, la estructura de los paquetes, el uso del puente `ros_gz_bridge` y la detección de marcadores fiduciales están respaldadas por la evidencia de cinco proyectos y repositorios oficiales de la comunidad robótica internacional:

Proyecto / Autor	Evidencia Técnica y Aporte a Nuestro Diseño	Enlace al Repositorio
1. TurtleBot3 Simulations (Rama Jazzy) — *ROBOTIS*	Demuestra cómo ROBOTIS migró el TurtleBot3 Waffle Pi de Gazebo Classic a Gazebo Harmonic en ROS 2 Jazzy. Evidencia el uso de sensores SDF puros sin `libgazebo_ros_camera` y el mapeo de tópicos mediante un archivo YAML centralizado (`turtlebot3_waffle_pi_bridge.yaml`) [4].	**github.com/ROBOTIS-GIT/turtlebot3_simulations**
2. ROS 2 + Gazebo Project Template — *Open Robotics*	Plantilla arquitectónica oficial que establece las buenas prácticas para separar la descripción del robot/mundo (SDF), la configuración del puente y los nodos de aplicación en proyectos modernos de ROS 2 + Gazebo Harmonic [6].	**github.com/gazebosim/ros_gz_project_template**
3. Paquete ros_gz (Bridge & Image) — *GazeboSim*	Documenta la evidencia de rendimiento y la sintaxis exacta para conectar flujos unidireccionales (`GZ_TO_ROS`) de cámaras simuladas (`sensor_msgs/msg/Image` y `CameraInfo`) optimizando el ancho de banda [2].	**github.com/gazebosim/ros_gz (README)**
4. Proyecto ros2_aruco — *James Madison University (JMU)*	Proyecto académico de robótica móvil que implementa la detección de marcadores ArUco en ROS 2 usando Python, `cv_bridge` y `OpenCV`, suscribiéndose simultáneamente a `/image_raw` y `/camera_info` para identificar objetivos [5].	**github.com/JMU-ROBOTICS-VIVA/ros2_aruco**
5. Gazebo Sensors (gz-sensors) — *Open Robotics*	Especificación técnica oficial de los modelos matemáticos de cámara en Gazebo Harmonic, validando los parámetros de campo visual (FOV), planos de recorte (*near/far clip*) y motor gráfico `ogre2` [3].	**gazebosim.org/libs/sensors**

5. Hoja de Ruta Progresiva: Cómo Construiremos el Proyecto Paso a Paso

Para asegurar un desarrollo ordenado, comprensible y defendible en cada entrega parcial, el proyecto se divide en cuatro fases incrementales donde ninguna etapa avanza sin haber validado la anterior [1]:

Fase / Avance	Enfoque Principal	Entregable y Meta de Comprensión
Fase 1 (Actual)	Fundamentación, Diseño Visual y Selección de Herramientas	Sustentación de la problemática, propuesta, selección y ubicación de la cámara RGB en Gazebo Harmonic, definición de librerías y análisis de proyectos de referencia.
Fase 2 (Próximo Paso)	Instrumentación de la Cámara, Puente ROS-GZ y Nodo Base de Visión	Instalación de dependencias en Ubuntu 24.04, puesta en marcha del sensor en Gazebo Harmonic, activación de `ros_gz_bridge` y primera detección ArUco/HSV en Python.
Fase 3	Escenario de 3 Estaciones y Navegación Secuencial (Nav2)	Ubicación de las estaciones P1, P2 y P3 en el mundo simulado, patrullaje autónomo con `goToPose`, control de orientación *yaw* y parada de estabilización frente al panel.
Fase 4 (Cierre)	Máquina de Estados, Votación Temporal y Reporte Auditable	Clasificación robusta por votación de 5 fotogramas, generación automática de reportes CSV/JSON con imágenes anotadas y evaluación mediante matriz de confusión.

6. Referencias Técnicas y Proyectos de Referencia

1. Curso de Robótica UNAL / Convenio UNSAAC (2026). *Revisión y Mejora de Propuesta 02: Robot autónomo de inspección con visión artificial — Guía de implementación ROS 2 Jazzy / Gazebo Harmonic*. Documento base aportado al proyecto, pp. 1–11.
2. Open Robotics / GazeboSim (2025). *ros_gz_bridge & ros_gz_image: Bridge communication between ROS 2 and Gazebo Transport*. Repositorio oficial en GitHub: **https://github.com/gazebosim/ros_gz/blob/ros2/ros_gz_bridge/README.md**
3. Open Robotics / GazeboSim (2025). *Gazebo Sensors Library (gz-sensors): CameraSensor and SDFormat specification*. Documentación oficial: **https://gazebosim.org/libs/sensors/**
4. ROBOTIS-GIT (2025). *turtlebot3_simulations: ROS 2 Jazzy packages for TurtleBot3 Gazebo Harmonic simulations*. Repositorio oficial: **https://github.com/ROBOTIS-GIT/turtlebot3_simulations**
5. JMU-ROBOTICS-VIVA — James Madison University (2024). *ros2_aruco: ROS 2 Wrapper for OpenCV ArUco Marker Tracking*. Repositorio de referencia: **https://github.com/JMU-ROBOTICS-VIVA/ros2_aruco**
6. Open Robotics / GazeboSim (2025). *ros_gz_project_template: A template project integrating ROS 2 and modern Gazebo simulator*. Repositorio oficial: **https://github.com/gazebosim/ros_gz_project_template**
7. OpenCV Team (2024). *Detection of ArUco Markers in OpenCV 4.x (cv::aruco Module)*. Documentación oficial: **https://docs.opencv.org/master/d9/d6d/tutorial_table_of_content_aruco.html**